

DISPENSA DELLA LEZIONE

Arrow function e metodi degli array

Concetto di base delle funzioni compatte, visibilità delle variabili e uso dei principali metodi sugli array.

CAPITOLO 1

Arrow function

NOTA

Una arrow function è una funzione scritta in forma compatta con `=>`. Ha gli stessi elementi logici di una funzione classica: parametri, corpo della funzione e, quando serve, un valore restituito.

I **parametri** sono i valori in ingresso che la funzione riceve. Nella funzione classica si scrivono tra parentesi tonde dopo `function`; nella arrow function si scrivono nello stesso modo, ma senza la parola `function`. Se il parametro è uno solo, le parentesi tonde non sono obbligatorie e si possono omettere. Se i parametri sono più di uno, le parentesi tonde restano necessarie. Se non ci sono parametri, le tonde sono comunque obbligatorie.

Il **corpo della funzione** è la parte che contiene le istruzioni da eseguire. Nella funzione classica il corpo sta dentro le graffe dopo le tonde. Anche nella arrow function si possono usare le graffe quando ci sono più istruzioni. Se l'istruzione è una sola, il `return` diventa implicito: il valore dell'espressione viene restituito automaticamente e non serve scrivere `return`. Se invece il corpo contiene più istruzioni e usi le graffe, allora `return` torna necessario per restituire un risultato.

`return` serve a restituire un valore dalla funzione. Nella funzione classica si scrive dentro il corpo quando vuoi dare un risultato all'esterno. Nella arrow function, se il corpo è una sola espressione, il ritorno è implicito e `return` non serve. Se invece usi le graffe, `return` torna necessario.

Esempio

```
function calcolaSconto(prezzo) {  
    return prezzo * 0.9;  
}  
  
const calcolaScontoArrow = prezzo => prezzo * 0.9;  
  
console.log(calcolaSconto(50));  
console.log(calcolaScontoArrow(50));
```

In questo esempio il parametro è **prezzo**. La funzione classica usa le tonde per il parametro, le graffe per il corpo e **return** per restituire il risultato. La arrow function scrive lo stesso parametro con sintassi più breve e, siccome c'è una sola espressione, il **return** è implicito.

CAPITOLO 2

forEach

NOTA

forEach esegue una funzione su ogni elemento dell'array, senza creare un nuovo array. Si usa quando vuoi fare un'azione per ogni valore, per esempio stampare tutti i prodotti di una lista.

forEach serve per percorrere un array e compiere un'azione su ogni elemento. Non restituisce un nuovo array, quindi è adatto quando vuoi solo leggere i valori, stamparli o aggiornare una variabile esterna. In un contesto reale può essere usato per mostrare una lista di articoli, studenti o ordini.

La callback riceve ogni elemento uno alla volta e il metodo la esegue fino alla fine dell'array. Questo lo rende utile per operazioni di controllo o visualizzazione, quando non serve trasformare i dati.

Esempio

```
const prodotti = ["penna", "quaderno", "zaino"];

prodotti.forEach(prodotto => {
  console.log(`Prodotto disponibile: ${prodotto}`);
});
```

CAPITOLO 3

map

NOTA

map crea un nuovo array con lo stesso numero di elementi dell'originale. Ogni valore viene trasformato dalla callback e salvato nel nuovo array.

map si usa quando vuoi trasformare ogni elemento di un array in un nuovo valore. Il risultato è sempre un nuovo array, utile quando devi modificare prezzi, formati o dati da mostrare in una pagina. L'array iniziale non viene cambiato.

Questo metodo è molto comune nei progetti reali, per esempio quando bisogna applicare un'imposta a una lista di prezzi o creare una versione più leggibile di un elenco di oggetti.

Esempio

```
const prezzi = [10, 20, 30];  
const prezziConIVA = prezzi.map(prezzo => prezzo * 1.22);  
  
console.log(prezziConIVA);
```

CAPITOLO 4

find

NOTA

`find` restituisce il primo elemento che soddisfa una condizione. Se non trova nessun elemento valido, restituisce `undefined`.

`find` serve quando devi cercare un solo elemento in un array. Controlla i valori uno alla volta e si ferma appena trova il primo che rispetta la condizione. È utile quando vuoi recuperare un prodotto specifico, un utente o un elemento con una certa caratteristica.

Dal punto di vista pratico, `find` è la scelta giusta quando non ti serve tutta la lista filtrata, ma solo il primo risultato utile.

Esempio

```
const prodotti = ["penna", "quaderno", "zaino"];  
const trovato = prodotti.find(prodotto => prodotto === "quaderno");  
  
console.log(trovato);
```

CAPITOLO 5

filter

NOTA

filter crea un nuovo array con solo gli elementi che rispettano la condizione. Controlla tutti i valori e conserva quelli validi.

filter si usa quando vuoi selezionare solo alcuni elementi di un array. Il risultato è un nuovo array che contiene esclusivamente i valori che superano il controllo. Questo metodo è perfetto per separare elementi validi, importanti o sopra una certa soglia.

In un caso reale puoi usarlo per tenere solo i prodotti disponibili, gli studenti promossi o gli ordini sopra un certo importo.

Esempio

```
const ordini = [12, 18, 7, 30];  
const ordiniValidi = ordini.filter(importo => importo >= 10);  
  
console.log(ordiniValidi);
```

CAPITOLO 6

some

NOTA

`some` restituisce `true` se almeno un elemento rispetta la condizione. Appena trova un valore valido, si ferma.

`some` serve per controllare se in un array esiste almeno un elemento che soddisfa una condizione. Il risultato è sempre un valore booleano: `true` o `false`. È utile quando ti basta sapere se qualcosa esiste, per esempio se c'è almeno un prodotto economico o un numero dispari.

Questo metodo è molto comodo nei controlli rapidi, perché evita di analizzare tutto l'array quando il primo elemento valido è già sufficiente.

Esempio

```
const prodotti = ["pasta", "sugo", "formaggio"];
const esisteProdottoSenzaGlutine = prodotti.some(prodotto => prodotto === "pasta");

console.log(esisteProdottoSenzaGlutine);
```

CAPITOLO 7

every

NOTA

`every` restituisce `true` solo se tutti gli elementi rispettano la condizione. Se anche un solo elemento non la rispetta, il risultato diventa `false`.

`every` si usa quando vuoi controllare che tutta la lista rispetti una regola. Restituisce un valore booleano e permette di capire subito se l'array è completamente valido. È utile, per esempio, per verificare che tutti i prodotti abbiano un prezzo positivo o che tutti gli studenti siano presenti.

A differenza di `some`, che si ferma al primo elemento valido, `every` richiede che la condizione sia vera per tutti gli elementi.

Esempio

```
const etàClienti = [18, 22, 31];  
const tuttiMaggiorenni = etàClienti.every(età => età >= 18);  
  
console.log(tuttiMaggiorenni);
```